

Autonomous Cognitive Agent System

1. Introduction

This project implements an Autonomous Cognitive Agent System designed to perform intelligent, long-horizon tasks through structured planning, reasoning, tool usage, and memory management. The system is built using Python, LangGraph, LangChain, and Groq-hosted Large Language Models (LLMs), with both command-line and Streamlit-based user interfaces.

The core idea is to move beyond single-prompt LLM usage and instead build an agentic workflow where tasks are planned, executed step by step, and stored in memory for future reference. The architecture is modular, extensible, and suitable for research and real-world automation.

2. Objectives

- Build an autonomous AI agent capable of reasoning and execution
- Implement structured task planning using TODO decomposition
- Overcome LLM context limitations using persistent memory
- Separate reasoning from execution using tools
- Enable supervisor-worker style coordination
- Provide an interactive user interface

3. Technology Stack

Programming Language

- Python

Agent & Workflow Frameworks

- LangGraph – Stateful agent execution
- LangChain – Tool abstraction and LLM utilities

Large Language Model

- Groq LLM (llama-3.1-8b-instant)

Utilities

- python-dotenv – Environment management
- LangSmith – Tracing and observability

User Interface

- Streamlit

4. System Architecture Overview

The system follows a Supervisor–Worker architecture implemented using LangGraph. Each agent operates on a shared state, enabling controlled execution and persistent context.

Key Principles:

- Explicit planning before execution
- Clear separation of reasoning and actions
- Tool-driven interaction with memory
- Stateful, explainable workflows

5. Environment Setup and LLM Initialization

The Groq LLM client is initialized using environment variables stored in a .env file.

```
import os
from dotenv import load_dotenv
from groq import Groq

load_dotenv()

API_KEY = os.getenv("GROQ_API_KEY")
if not API_KEY:
    raise ValueError("GROQ_API_KEY not found. Please check your .env file.")

client = Groq(api_key=API_KEY)
```

Reusable LLM call wrapper:

```
def llm_call(messages):
    response = client.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=messages
    )
    return response.choices[0].message.content
```

6. Task Planning Using TODO Decomposition

User requests are converted into structured TODO steps to enable deterministic execution.

```
from langchain.tools import tool

@tool
def write_todos(request: str):
    """Break a user request into small, clear TODO steps."""
    system = "You are a task planner. Break the request into small, clear TODO steps."
    msg = [
        {"role": "system", "content": system},
        {"role": "user", "content": request}
    ]
    result = llm_call(msg)
    todos = [
        {"task": t.strip().lstrip("0123456789.- ")
         for t in result.split("\n") if t.strip()}
    ]
    return todos or []
```

7. Supervisor Agent

The Supervisor Agent interprets user intent, generates plans, and manages execution state.

```
def supervisor_node(state):
    if not state.get("todos"):
        todos = write_todos.invoke(state["user_query"])
```

```
state["todos"] = todos or []
state["current_step"] = 0
return state
```

8. Worker Agent

The Worker Agent executes individual TODO tasks sequentially.

```
def worker_node(state):
    if state["current_step"] >= len(state.get("todos", [])):
        return state

    task = state["todos"][state["current_step"]]["task"]

    messages = [
        {"role": "system", "content": "You are a helpful worker agent."},
        {"role": "user", "content": task}
    ]

    result = llm_call(messages)
    state["todos"][state["current_step"]]["result"] = result
    state["current_step"] += 1
    return state
```

9. Workflow Orchestration with LangGraph

```
from langgraph.graph import StateGraph, END

workflow = StateGraph(dict)
workflow.add_node("supervisor", supervisor_node)
workflow.add_node("worker", worker_node)
workflow.set_entry_point("supervisor")
workflow.add_edge("supervisor", "worker")
workflow.add_conditional_edges(
    "worker",
    lambda state: END if state["current_step"] >= len(state.get("todos", [])) else
    "worker",
    {"worker": "worker", END: END}
)
```

```
graph = workflow.compile()
```

10. Persistent Memory – Virtual File System (VFS)

The Virtual File System allows agents to store and retrieve information beyond the LLM context window.

```
from langchain_core.tools import tool

@tool
def write_file(filename: str, content: str, vfs: dict) -> str:
    vfs[filename] = content
    return f'File '{filename}' written successfully.'

@tool
def read_file(filename: str, vfs: dict) -> str:
    return vfs.get(filename, "File not found.")

@tool
def list_files(vfs: dict) -> str:
    if not vfs:
        return "No files present."
    return "\n".join(vfs.keys())
```

11. Streamlit User Interface

```
import streamlit as st
from agent import graph

st.set_page_config(page_title="AI Agent", layout="centered")
st.title("🧠 Intelligent AI Agent")
```

The UI provides:

- Real-time chat interaction
- Persistent session state

- Clear visualization of agent responses

12. End-to-End Execution Flow

1. User submits a query
2. Supervisor generates a TODO plan
3. Worker executes tasks sequentially
4. Memory is updated when needed
5. Final response is generated
6. Output is displayed to the user

13. Setup and Usage

Prerequisites

Before running the project, ensure the following requirements are met:

- Python 3.10 or above installed
- A valid Groq API Key
- Internet connectivity for LLM access

Environment Setup

1. Clone the project repository or extract the project folder.
2. Create a virtual environment (recommended):

```
python -m venv venv  
source venv/bin/activate # On Windows: venv\Scripts\activate
```

3. Install required dependencies:

```
pip install -r requirements.txt
```

4. Create a .env file in the project root and add your Groq API key:

```
GROQ_API_KEY=your_api_key_here
```

- Running the Agent (CLI Mode)

5. To start the agent in command-line mode:

```
python agent.py
```

- You can then interact with the agent by typing queries directly into the terminal. Type exit or quit to stop the session.
- Running the Streamlit Application

6.To launch the web-based interface:

```
streamlit run app.py
```

This opens a browser window where users can chat with the autonomous agent in real time. The interface maintains session state and displays responses clearly.

Typical Usage Flow

1. User submits a query
2. Supervisor agent generates a TODO-based plan
3. Worker agent executes tasks sequentially
4. Memory is updated when required
5. Final response is generated and displayed

13. Conclusion

This project demonstrates a scalable and explainable autonomous agent architecture that integrates planning, reasoning, memory, and execution. The modular design allows easy extension with additional agents, tools, and workflows, making it suitable for advanced AI research and practical automation use cases.